گزارش جامع بهینه‌سازی

اجرای CPU مدل GLM-5.3-Flash

از موتور اولیه تا سرور آزمایشی امروز

مسیر اعتبارسنجی‌شدهٔ چهار فاز: AMX و NUMA، عملیات گراف، MTP، و کِرنل‌های Prefill

سرور: 1 TiB RAM | 2× Intel Xeon Platinum 8480+ | HPE DL380 Gen11 — g11 | بدون GPU

مدل: GLM-5.3-Flash UD-Q4_K_XL (GGUF شش‌پاره، ۱۸۶ گیابایت)

۲۰ مهر ۱۴۰۵ — 12 October 2026

فهرست مطالب

۱. خلاصهٔ مدیریتی

۲. سخت‌افزار، مدل و وضعیت اولیه

۳. روش‌شناسی اعتبارسنجی

۴. مسیر بهینه‌سازی در یک نگاه

۵. فاز ۱ — اصلاحات پایه: AMX، دیسپچ دسته‌ای و NUMA

۶. فاز ۲ — عملیات گراف: CONCAT و GET_ROWS

۷. فاز ۳ — MTP و کِرنل‌های VNNI

۸. فاز ۴ — کِرنل‌های Prefill و عبور از ۲۰۰

۹. دقت، صحت و بحث کوانتیزاسیون

۱۰. زیرساخت فعلی: سرورها، کش پرامپت و کانتکست بزرگ

۱۱. ابزار تست: رابط کاربری

۱۲. جمع‌بندی و آنچه باقی مانده است

۱۳. پیوست — بازتولید، فایل‌ها و سابقهٔ تغییرات

۱. خلاصهٔ مدیریتی

این گزارش شرح کامل و مستندِ مسیر بهینه‌سازی اجرای CPU مدل GLM-5.3-Flash روی سرور g11 است: از موتور اولیهٔ llama.cpp با سرعت ۵۷.۶ توکن بر ثانیه در prefill و ۶.۲ توکن بر ثانیه در decode، تا وضعیت فعلی با ۲۰۶.۷ توکن بر ثانیه prefill (اعتبارسنجی‌شده؛ تا ۲۳۶ در اندازه‌گیری زنده) و ۱۷.۷ تا ۱۸.۱ توکن بر ثانیه decode. نکتهٔ محوری این مسیر آن است که هر بهینه‌سازی پیش از پذیرش، با تست‌های عددی بایت‌به‌بایت و مقایسهٔ خروجی کامل مدل روی پرامپت‌های واقعی راستی‌آزمایی شده و در صورت نبودِ سود، صریحاً رد و ثبت شده است.

بهبود کلی در طول مسیر:

سنجه	موتور اولیه	وضعیت فعلی	ضریب بهبود
Prefill — پرامپت واقعی انگلیسی (۱۸۶۰ توکن)	57.6*	206.7 (زنده: تا 236.3)	3.6× تا 4.1×
Decode — پرامپت واقعی انگلیسی	6.2*	17.7 – 18.1	≈2.9×
Prefill گرم با کش پرامپت (نوبت دوم مکالمه)	—	0.89s به‌جای 8.36s	9.4×
کانتکست هر اسلات	8192	131072	16×

* اعداد «موتور اولیه» از بنچمارک متن تکراری ۵۱۲/۲۰۴۸ توکنی است که تنها مبنای مقایسهٔ آن دوره است؛ زنجیرهٔ پرامپت واقعی از ردیف‌های فاز ۱ در بخش ۴ آغاز می‌شود.

در کنار موتور، یک رابط کاربری فارسی تستِ سرعت ساخته شد که استریم زنده، نمایش تفکر مدل، متریک‌های Prefill/Decode/MTP/کش و حالت حافظهٔ مکالمه را ارائه می‌دهد و تمام سنجه‌های این گزارش را از مرورگر قابل اندازه‌گیری می‌کند.

۲. سخت‌افزار، مدل و وضعیت اولیه

تمام کارها روی سرور g11 (saeed@192.168.20.126) انجام شده است: سرور HPE DL380 Gen11 با دو پردازندهٔ Intel Xeon Platinum 8480+ (مجموعاً ۱۱۲ هستهٔ فیزیکی، با پشتیبانی Intel AMX)، ۱ ترابایت RAM و بدون هیچ GPU. کل محاسبات مدل به‌طور کامل روی CPU و با تراشه‌های ماتریسی AMX انجام می‌شود.

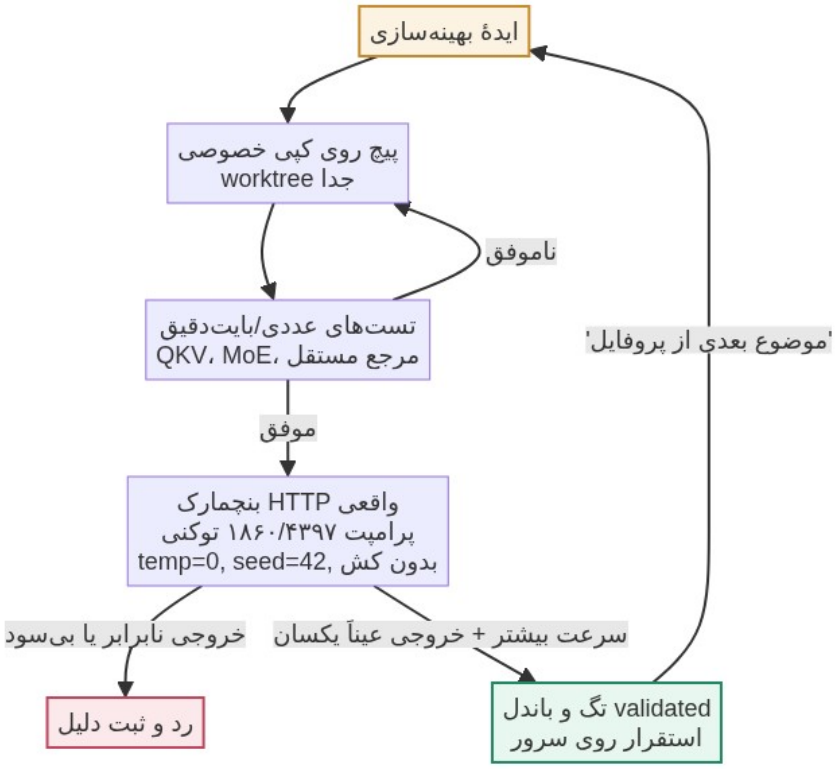
جزء	مشخصات
سرور	HPE DL380 Gen11 — دو سوکت
پردازنده	2× Intel Xeon Platinum 8480+ — ۱۱۲ هستهٔ فیزیکی، پشتیبانی AMX
حافظه	1 TiB DDR5 (NUMA دوگره‌ای)، بدون GPU
مدل	GLM-5.3-Flash UD-Q4_K_XL — حدود GGUF، شش شارد ۱۸۶ گیابایت
موتور	llama.cpp (کپی خصوصی روی سرور) — ساخت Release، GGML_NATIVE + GGML_AMX، بدون OpenMP
مدل جایگزین موجود	Q8E (۲۸۱ گیابایت) — تا امروز بارگذاری نشده برای مقایسهٔ دقت کوانتیزاسیون؛

وضعیت اولیه

در نقطهٔ شروع، موتور اصلی بدون هیچ تغییری این ویژگی‌ها را داشت: اجرای MoE بدون پشتیبانی بافر AMX (سقوط به کِرنل‌های CPU عمومی)، ثردینگ ۳۲/۵۶، بدون بازپک وزن‌ها، بدون جانمایی NUMA و با دیسپچ دسته‌ای معیوب. تنها مبنای عددی آن دوره، بنچمارک متن تکراری انگلیسی بود: prefill ۵۷.۶۰ و decode ۶.۲۰ توکن بر ثانیه. هدف اعلامی اولیه (prefill ۱۰ / decode ۵۰ روی متن تکراری) به‌سرعت پاس شد، اما تست روی پرامپت‌های واقعی نشان داد هدف واقعی باید بسیار بالاتر باشد؛ به همین دلیل زنجیرهٔ اصلی گزارش بر «پرامپت‌های واقعی» بنا شد: یک پرامپت انگلیسی ۱۸۶۰ توکنی و یک پرامپت فارسی ۴۳۹۷ توکنی (بازبینی کد واقعی از سورس موتور)، همیشه با seed=42، temperature=0، بدون کش پرامپت و تک‌درخواست.

۳. روش‌شناسی اعتبارسنجی

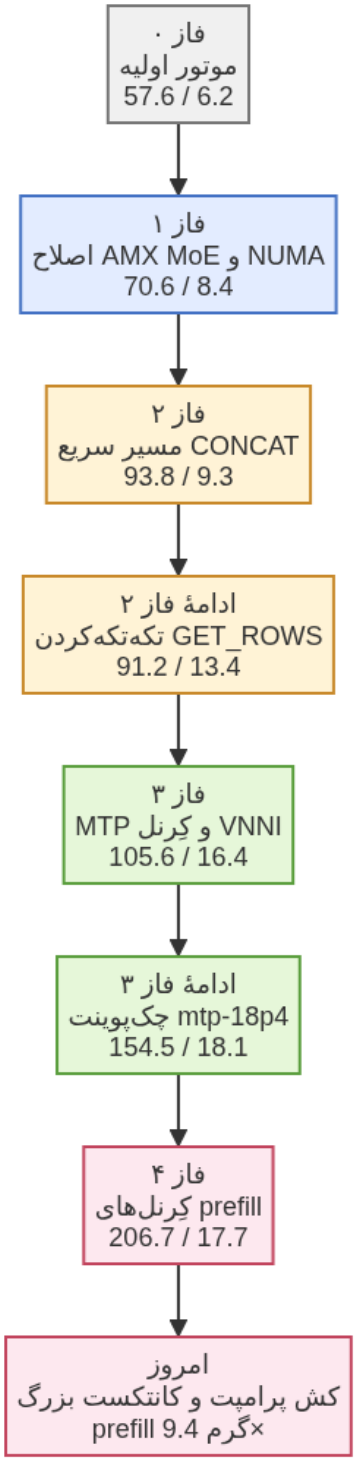
قاعدهٔ حاکم بر تمام مسیر این بود: «هیچ تغییری بدون اثبات صحت و سود واقعی ادغام نمی‌شود.» هر ایده از حلقهٔ زیر عبور می‌کرد:



شکل ۱ — حلقهٔ اعتبارسنجی هر بهینه‌سازی

- **تست عددی بایت‌دقیق** — هر کِرنل در برابر مرجع میزبان (و در مواردی مرجع مستقل بایت‌به‌بایت) تست می‌شد؛ مثلاً ۳۸۴ مورد QKV، ۷۲ مورد MoE، ۱۲۸ مورد GET_ROWS و ۱۹۲ مورد fuse گیت.
- **اعتبارسنجی HTTP واقعی** — دو پرامپت واقعی ثابت با seed و دمای صفر و بدون کش؛ پذیرش تنها با «خروجی کامل عیناً برابر» چک‌پوینت قبلی مجاز بود.
- **نسخه‌بندی و باندل** — هر چک‌پوینت موفق با تگ و باندل Git محفوظ شد (validated-real-8p6 تا validated-real-mtp-18p4) تا همیشه امکان بازگشت باشد.
- **بی‌اعتمادی به میکروبنچمارک** — بهبود میکروبنچمارک به‌تنهایی هرگز ملاک پذیرش نبود؛ چند کِرنل با بنچمارک بهتر، روی درخواست واقعی کندتر بودند و رد شدند.
- **ایزوله‌سازی و پروفایل** — هر آزمایش در worktree جداگانه اعمال و به‌صورت patch ذخیره شد؛ پروفایل پیش از هر فاز، مسیر بعدی را تعیین می‌کرد.

۴. مسیر بهینه‌سازی در یک نگاه



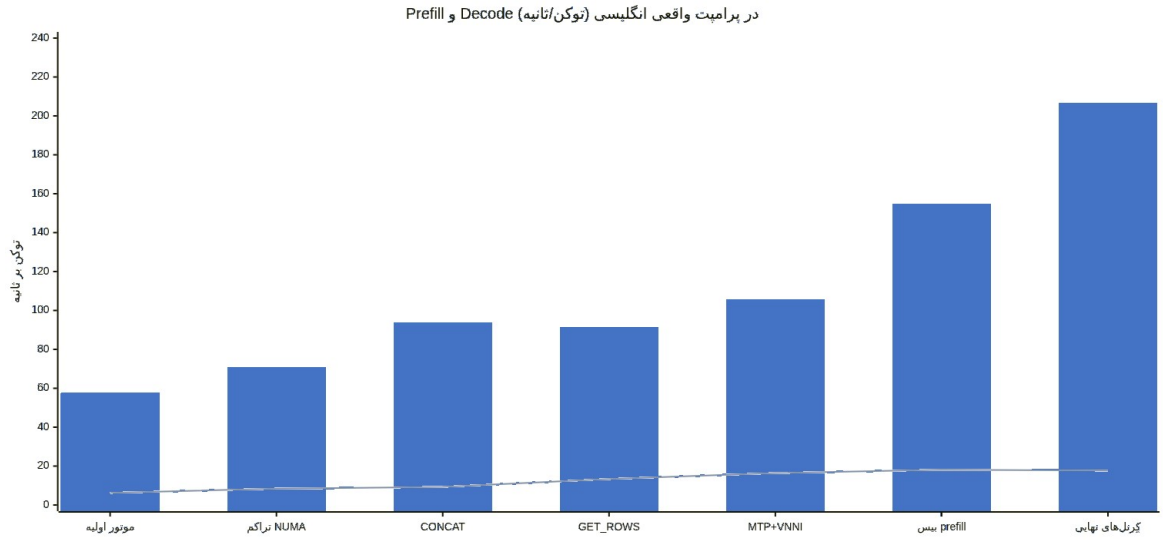
شکل ۲ — چهار فاز مسیر با نرخ‌های اعتبارسنجی‌شده (توکن بر ثانیه، پرامپت واقعی انگلیسی)

جدول زیر زنجیرهٔ کامل چک‌پوینت‌های اعتبارسنجی‌شده روی پرامپت واقعی است. همهٔ اعداد توکن بر ثانیه، تک‌درخواست، بدون کش و با خروجی کامل عیناً برابر چک‌پوینت قبلی‌اند:

چک‌پوینت / تگ	تغییر کلیدی	EN: Prefill	EN: Decode	FA: Prefill	FA: Decode

MTP7، p_min=0.7	وضعیت پیش از اصلاحات (MTP قدیمی)	64.67	5.32	64.20	5.61
(NUMA پیش از) —	MTP خاموش‌کردن قدیمی	69.46	7.26	68.18	7.45
validated-real-8p6	NUMA اصلاح dense (فاز ۱)	70.55	8.43	68.66	8.59
validated-real-9p4 (c9b8ee9)	مسیر سریع CONCAT (فاز ۲)	93.83	9.26	89.03	9.39
validated-real-13p3 (d079289)	تکه‌تکه‌کردن GET_ROWS (فاز ۲)	91.18	13.35	86.47	13.10
prefill-100 (f4e9512)	MTP + پایدار VNNI دوم‌سطری (فاز ۳)	105.58	16.37	103.75	16.23
validated-real-mtp-18p4 (1a1d1ab)	تکمیل فاز ۳ — بیس فاز ۴	154.54	18.09	147.25	17.82
prefill-final	کِرنل‌های Q8 signed و Q4/Q5 scaled-K (فاز ۴)	206.66	17.68	206.05	17.51
اجرای زندهٔ امروز (8017)	همان پیکربندی، سرور خلوت	221.7 – 236.3	18.1 – 18.3	—	—

نمودار زیر همان زنجیره را نشان می‌دهد — میله‌ها prefill و خط پُررنگ‌تر decode است:



شکل ۳ — پیشرفت prefill و decode در طول مسیر

دو نکتهٔ صداقت علمی: نخست اینکه فاصلهٔ چک‌پوینت‌های mtp-16p3 و mtp-18p4 (پرش decode از ۱۶.۴ به ۱۸.۴) جزئیات مستند در فایل‌های محلی ندارد و تنها باندل آن محفوظ است. دوم اینکه اعداد مطلق بین روزها تا ~ ۱۰٪ (و در یک مورد بیشتر) نوسان می‌کنند؛ به همین دلیل مقایسه‌های معتبر فقط «هم‌جلسه» تلقی شده‌اند (بخش ۸).

۵. فاز ۱ — اصلاحات پایه: AMX، دیسپچ دسته‌ای و NUMA

راه‌اندازی AMX و رفع دیسپچ

نخستین گام، اثبات سلامت سخت‌افزار بود: یک تست مستقل INT8 مجوز XTILEDATA را گرفت، پیکربندی تایل را ساخت و هر ۲۵۶ خروجی را در برابر مقدار مرجع ۳۸۴ راستی‌آزمایی کرد — نتیجه: AMX سالم است و نیازی به تغییر BIOS نیست. سپس پشتیبانی بافر AMX برای ضرب MoE اضافه شد، تخصیص ماتریس‌های expert/batched اصلاح شد و باگ‌های بایت/المان و استرید اکتیویشن در مسیر batched برطرف گردید. عملیاتِ وزن‌های پک‌شدهٔ پشتیبانی‌نشده به‌جای سقوط به کِرنل خام، صریحاً رد می‌شود. دیسپچ FP16 روی AMX به‌دلیل عبور از حد تحمل خطا غیرفعال ماند.

جانمایی NUMA و ثردینگ

پروفایل decode روی درخواست واقعی انگلیسی نشان داد ۵۹٪ نمونه‌های چرخهٔ CPU در نواربار (barrier) و ۱۹.۷٪ در کِرنل برداری Q8_0 است — یعنی گلوگاه اصلی، دسترسی‌های دور در سیستم دو سوکتی است. راه‌حل: نیمه‌های ردیف خروجی هر expert و ماتریس‌های dense/batched به گرهٔ NUMA همان سوکت متصل شد و هر worker به هستهٔ فیزیکی پین شد؛ نواربار دوگروهی با همگام‌سازی acquire/release و فعال‌سازی FTZ/DAZ برای نخ‌های worker. نتیجه: decode واقعی ۱۵–۱۶٪ بهبود یافت (۷.۲۶ → ۸.۴۳ انگلیسی) با خروجی عیناً برابر.

پیکربندی آن دوره (متن تکراری)	ورودی	Prefill	Decode
— موتور اولیه بدون بازپک t32/tb56	512	57.60	6.20
همان — ورودی بزرگ‌تر	2048	56.84	6.10
— تک‌سوکت AMX MoE t24/tb56، ub2048	512	65.50	7.75
دسته‌ای + ردیف‌های AMX محلی expert + MTP7/p0	2048	67.14	11.43

- **ردشدهٔ این فاز:** — کِرنل MoE عریض‌تر: تست عددی را پاس کرد اما روی درخواست واقعی کندتر بود (۷.۹۲) — رد شد.
- تقسیم جمع صحیح Q8 به زنجیره‌های مستقل: تست پاس، عملکرد واقعی بدتر — رد شد.
- یک آزمایش دیسپچ ناایمنِ ابتدایی متن خراب تولید کرد و فوراً کنار گذاشته شد.

۶. فاز ۲ — عملیات گراف: CONCAT و GET_ROWS

CONCAT موازی

پروفایل زمان دیوار decode نشان داد سهم بزرگی از زمان در CONCAT و GET_ROWS است. مسیر سریع F32/I32 برای CONCAT ردیف‌های به‌هم‌پیوسته نوشته شد: تقسیم تخت ردیف‌ها بین worker‌ها و کپی

سگمنت‌ها. هر ۳۲ تست صحت (ابعاد ۰ تا ۳، ۱/۳/۸/۴۸ ثرد، ورودی پیوسته و جابه‌جاشده) پاس شد. نتیجهٔ واقعی: انگلیسی از ۷۰.۶ به ۹۳.۸ prefill و از ۸.۴ به ۹.۳ decode (تگ validated-real-9p4، کامیت c9b8ee9) — پس از بازگشت شبکهٔ سرور که مستندسازی آن در CHECKPOINT.md آمده است.

GET_ROWS تکه‌تکه‌شده — جهش decode به +۱۳

پروفایل‌بندی نشان داد هر توکن decode حدود ۳۲.۴ میلی‌ثانیه فقط در GET_ROWS صرف می‌کند (۳۱.۸ میلی‌ثانیه آن محاسبهٔ واقعی نخ ۰، نه انتظار نواربری) — صفحات حالت بازگشتی با ردیف‌های تا ۱,۰۴۸,۵۷۶ مقدار. مسیر GGML_ROWS_CHUNKS بلوک‌های ستونی ۱۰۲۴تایی پیوسته را بین workerهای گراف پخش می‌کند. هر ۱۲۸ تست مرجع مستقل بایت‌دقیق پاس شد (ردیف‌های میلیون‌عضوی، F32/I32، ثردینگ‌های مختلف، تانسورهای جابه‌جاشده). نتیجه: decode انگلیسی ۹.۳ → ۱۳.۳۵ و فارسی به ۱۳.۱۰ — عبور از هدف ۱۰ توکن بر ثانیه روی پرامپت واقعی (تگ validated-real-13p3، کامیت d079289).

- **آزمایش مکمل:** — fuse SCALE→SIGMOID→SCALE با ۱۹۲ تست بایت‌دقیق پاس شد اما روی پرامپت واقعی سودی نشان نداد (۱۳.۲۷/۱۳.۰۶) و فقط به‌صورت opt-in با FUSE_GATE=1 باقی ماند.

۷. فاز ۳ — MTP و کِرنل‌های VNNI

پس از عبور decode از ۱۳، هدف بعدی ۲۰ توکن بر ثانیه شد و مسیر اصلی، کدگذاری پیش‌گویانهٔ MTP (سر حد مدل با هد NextN) و کِرنل‌های فشرده‌تر شد. نکتهٔ مهم این فاز، جداسازی «پیکربندی» از «کِرنل» بود: همان MTP با پیکربندی اشتباه (p_min=0، cap=1، استخر 96 batched برای وارسی) فقط ۶.۸ توکن بر ثانیه می‌داد و رد شد؛ اما با استخرهای پایدار اختصاصی، وارسی موازی و زنجیره‌های کوچک به ۱۵.۱ رسید.

مرحله	تغییر	EN Prefill	EN Decode	MTP پذیرش
مبنا (بدون MTP)	چک‌پوینت p3	91.18	13.35	—
پیکربندی ردشده MTP	استخر p0، cap1، batched96	98.43	6.84	76–77٪
استخر پایدار MTP	+ استخرهای اختصاصی موازی نواری + CPY زنجیرهٔ کوچک	105.27	15.09	158/208
دوم‌سطری VNNI	اشتراک بار وزن بین دو ردیف اکتیویشن + آستانهٔ 8192	105.58	16.37	158/208
چک‌پوینت‌های بعدی	و سپس mtp-16p3 mtp-18p4 (1a1d1ab)	154.54*	18.09	—

* عدد ۱۵۴.۵ «بیس‌لاین تازه» روی همان کد mtp-18p4 با کِرنل‌های prefill خاموش است که مبنای سنجش فاز ۴ شد.

- **تغییرات مکمل این فاز:** — fuse جفت QKV و جفت gate/up با فعال‌ساز مشترک (۹۶ تست کوچک و ۲۴ تست بزرگ بایت‌دقیق).
- چیدمان interleave برای K و Q8 و مسیر small-batch تا ۸ توکن روی استخر decode.
- VNNI تک‌ردیفی برای decode عادی کندتر بود و رد شد؛ نسخهٔ دوم‌سطری ماندگار شد.
- **خلأ مستندسازی:** — از ۱۶.۴ تا ۱۸.۴: جزئیات این جهش فقط در باندل mtp-18p4 محفوظ است و در اسناد محلی شرح داده نشده است.

۸. فاز ۴ — کِرنل‌های Prefill و عبور از ۲۰۰

در این فاز فقط یک فایل تغییر کرد (mmq.cpp در worktree جداگانهٔ prefill-400) و همهٔ تغییرات به بلاک‌های ماتریسی با بیش از دو سطر محدود شد تا مسیر decode دست‌نخورده بماند. دو ایدهٔ اصلی حفظ شدند:

- **۱) GGML_Q8_PREFILL_SIGNED:** — وزن‌های Q8 پک‌شدهٔ unsigned یک‌بار به «اسکرچ signed» تبدیل می‌شوند و جبرانِ تکراری هر سطر از حلقهٔ درونی حذف می‌شود؛ مانع‌های کامپایلری خواندن اسکرچ را برای tile intrinsics صریح می‌کنند.
- **۲) GGML_PREFILL_SCALED_K:** — مقیاس‌های صحیح روی بایت‌پلن‌های کم/زیاد اعمال می‌شود، بلاک کامل ۲۵۶ عضوی با AMX جمع می‌شود و سپس ترتیب اصلی جمع اعشاری و min-correction حفظ می‌گردد — در نتیجه خروجی دقیقاً با مسیر مرجع یکی است.

همراه این دو: batch/ubatch=4096 و ۱۱۲ ثرد prefill (سوییپ‌ها نشان دادند ۵۶ و ۲۲۴ ثرد و batchهای ۱۰۲۴/۲۰۴۸ ضعیف‌ترند). صحت: ۳۸۴ تست QKV/raw-extreme، ۷۲ تست MoE و ۱۲ تست MoE بزرگ، همه بایت‌به‌بایت. نتیجه روی پرامپت واقعی: انگلیسی ۱۵۴.۵ → ۲۰۶.۷ (+۳۳.۷٪) و فارسی ۱۴۷.۳ → ۲۰۶.۱ (+۳۹.۹٪) با decode تقریباً دست‌نخورده (۱.۷٪- تا ۲.۳٪-). هدف ۴۰۰ محقق نشد و صادقانه در اسناد ثبت شده است.

ablation زندهٔ فلگ‌های کِرنل (همین هفته)

برای پاسخ به «سهم هر فلگ چقدر است»، هر چهار ترکیب پشت‌سرهم در یک جلسه اندازه‌گیری شد (ری‌استارت کامل سرور برای هر ترکیب، دو ران در هر ترکیب، ران دوم ملاک به‌دلیل گرم‌شدن ~۸٪):

ترکیب فلگ‌ها	توکن/ثانیه (ران گرم) Prefill	اختلاف
هر دو خاموش (کِرنل مرجع)	214.8	—
فقط Q8 signed	226.5	+5.4%
فقط Q4/Q5 scaled-K	224.1	+4.3%
هر دو (validated پیکربندی)	236.3	+9.9%

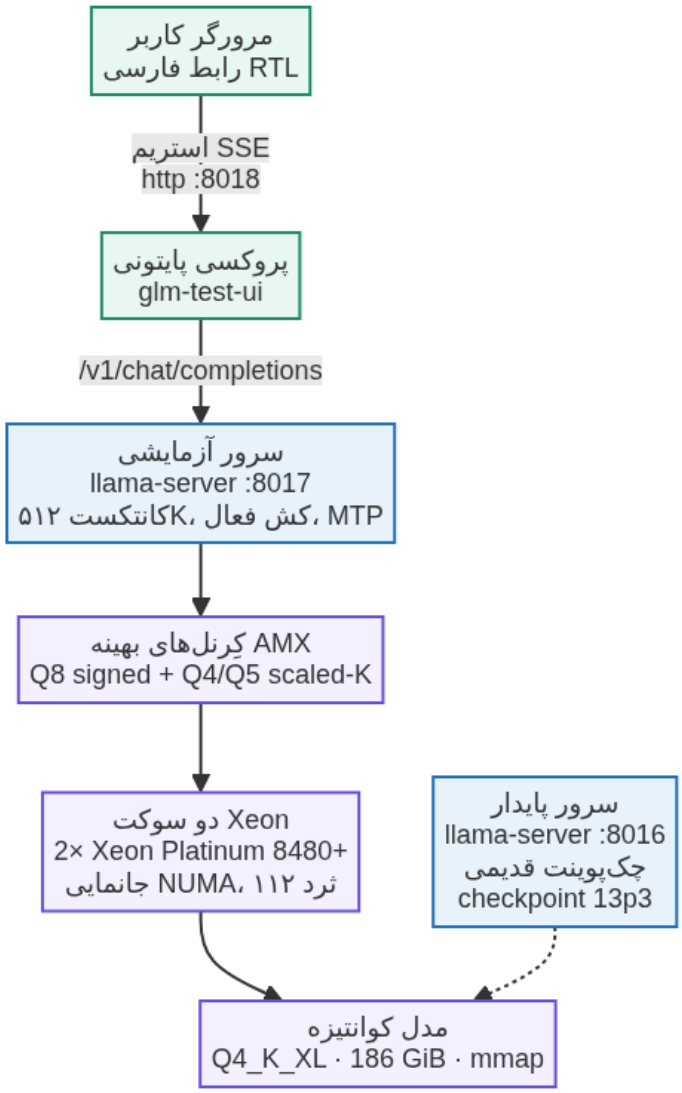
دو درس مهم: سهم دو فلگ جمع‌شدنی و در حد نویز (~۲ توکن) جداشدنی است؛ و اعداد مطلق بین جلسات نوسان بزرگی دارند — بیسِ بی‌کِرنل در همان جلسه ۲۱۴.۸ گرفت در حالی که در جلسهٔ ثبت اصلی ۱۵۴.۵ بود. بنابراین مقایسهٔ معتبر فقط هم‌جلسه است و این هشدار در PREFILL-RESULTS.md ثبت شده است.

۹. دقت، صحت و بحث کوانتیزاسیون

پرسش طبیعی: آیا Q4 بودنِ وزن‌ها دقت را می‌اندازد و آیا کِرنل‌های تازه چیزی را خراب کردند؟ پاسخ دقیـق دو لایـه دارد:

- **کِرنل‌ها صفر تغییر دقت:** — تمام کِرنل‌های تازه از نظر ریاضی همان مسیر مرجع را محاسبه می‌کنند (ترتیب جمع و تصحیح‌ها حفظ شده) و خروجی بایت‌به‌بایت با مرجع یکی است. مجموع تست‌های صحت در طول مسیر: ۳۸۴+ ۲۴۰ fuse، ۱۹۲ CONCAT، ۳۲ GET_ROWS، ۱۲۸ MoE، ۲×۲۴+۱۲+۷۲ QKV، گیت، ۱۹۲ زنجیرهٔ کوچک، ۸۰ batched و ۴۶۸ مورد فاز prefill.
- **کوانتیزاسیون مدل:** — مدل GLM-5.3-Flash از ابتدا با کوانت UD-Q4_K_XL (~۴.۵ بیت به‌ازای وزن) به کار گرفته شده است. این انتخاب lossy ذاتی خودش را دارد اما در این پروژه هرگز در برابر Q8 اندازه‌گیری نشده است؛ آنچه ثابت شد فقط «تغییرنکردنِ خروجی پس از بهینه‌سازی» است، نه «برابری Q4 با Q8». روی سرور نسخهٔ Q8E (۲۸۱ گیابایت) موجود است و می‌تواند مبنای یک مطالعهٔ مقایسه‌ای واقعی باشد.
- **شاهد عملی:** — برای هر تغییر، متن کامل پاسخ دو پرامپت واقعی با چک‌پوینت قبلی مقایسه می‌شد و در صورت کوچک‌ترین تفاوت، تغییر پذیرفته نمی‌شد؛ فیلد exact_complete_output در فایل‌های نتیجه این را ثبت می‌کند.

۱۰. زیرساخت فعلی: سرورها، کش پرامپت و کانتکست بزرگ



شکل ۴ — معماری استقرار فعلی روی g11

ویژگی	سرور پایدار :8016	سرور آزمایشی :8017
چک‌پوینت کد	validated-real-13p3 (d079289)	prefill-final (1 بیس a1d1ab + patch)
Prefill / Decode واقعی	≈13.4 / ≈91	≈18.3–17.7 / ≈206 (زنده تا ۲۳۶)
کانتکست	32768 (۴ ۸۱۹۲ × اسلات)	524288 (۴ ۱۳۱۰۷۲ × اسلات)
ترد decode / prefill	48 / 56	64 / 112
کش پرامپت	خاموش	روشن (--no-cache-prompt حذف شد)
کِرنل‌های prefill	—	Q8 signed + Q4/Q5 scaled-K
MTP	خاموش	draft-mtp، cap=1، p_min=0

کش پرامپت — اندازه‌گیری واقعی

نوبت	توکن پرامپت	کش‌شده	زمان prefill	توضیح
نوبت ۱ (سرد)	1854	0	8.36s	prefill کامل — ۲۲۱.۷ توکن/ثانیه
نوبت ۲ (گرم)	1868	1850 (۹۹٪)	0.89s	فقط ۱۸ توکن جدید پردازش شد

یعنی ادامهٔ مکالمه ۹.۴ برابر سریع‌تر پاسخ می‌دهد و نرخ decode دست‌نخورده می‌ماند. توجه: در حالت کش، متریک «توکن بر ثانیه» prefill معنا نمی‌دهد (۱۸ توکنِ باقیمانده همگی سربارند)؛ سنجهٔ درست، صرفه‌جویی زمان جدول بالا است. افزایش کانتکست به ۵۱۲ K هیچ رشد معناداری در مصرف RAM نداد (ساختار hybrid GDN مدل، کش حالت فشرده‌ای دارد) و پس از تخصیص واقعی هم ثابت ماند.

فلگ‌های محیطی فعال سرور 8017

run-prefill-400.sh (قابل بازتولید، در پیوست):

گروه	فلگ‌ها
AMX / NUMA	GGML_AMX_MOE، GGML_AMX_NUMA_ROWS، GGML_AMX_NUMA_DENSE، GGML_NUMA_PIN، GGML_CPU_FTZ، GGML_TREE_BARRIER
گراف	GGML_CONCAT_CHUNKS، GGML_ROWS_CHUNKS، GGML_COPY_ROWS
کِرنل‌ها	GGML_VNNI_ROWS2، GGML_K_INTERLEAVE، GGML_Q8_INTERLEAVE، GGML_FUSE_QKV، GGML_FUSE_MOE_PAIR، GGML_Q8_PREFILL_SIGNED، GGML_PREFILL_SCALED_K
مسیر کوچک / MTP	GGML_MTP_SHARED_POOL، GGML_CPU_SMALL_CHAIN(+MAX=8192)، GGML_SMALL_BATCH_MAX=8، LLAMA_MTP_ADAPTIVE=0
کانتکست و کش	CTX=524288 (پیش‌فرض 32768)، کش فعال (برای خاموش‌کردن NO_CACHE_PROMPT=1)

۱۱. ابزار تست: رابط کاربری

برای اینکه همهٔ این سنجه‌ها بدون اسکریپت، از مرورگر و قابل استفاده باشند، یک رابط فارسی RTL (GLM CPU Lab) ساخته و روی سرور مستقر شد (پورت 8018، پروکسی به 8017). قابلیت‌ها پس از تکمیل در این دوره:

- استریم زندهٔ SSE توکن‌به‌توکن با بج tok/s لحظه‌ای، بدون بافر شدن در پروکسی.
- نمایش «فرایند تفکر مدل» (reasoning_content) در بخش جمع‌شونده — پیش‌تر پاسخ‌های فقط‌استدلالی خالی نمایش داده می‌شدند.
- کارت‌های متریک: Prefill، Decode، توکن ورودی/خروجی، نرخ پذیرش MTP (draft_n_accepted/draft_n) و توکن کش‌شده.
- حالت «حافظهٔ گفتگو»: ارسال تاریخچهٔ مکالمه با cache_prompt=true برای سنجش سرعت مکالمهٔ پیوسته + دکمهٔ «گفتگوی جدید».
- حداکثر خروجی تا ۱۰۰,۰۰۰ توکن برای تست بلندمدت حالت استدلال (هماهنگ با کانتکست ۱۳۱ K هر اسلات).
- تاریخچهٔ ۵۰ اجرای آخر در مرورگر + نمودار روند + خروجی JSON + توقف با حفظ خروجی ناقص.

خود UI هم یک لایهٔ اندازه‌گیری است: اعداد جدول کش در بخش ۱۰ مستقیماً از همین مسیر (پروکسی 8018) گرفته شد و استریم آن به‌صورت تدریجی راستی‌آزمایی شد (۴۷ چانک در ۳.۷ ثانیه، بدون جهش بافری).

۱۲. جمع‌بندی و آنچه باقی مانده است

هدف	وضعیت	بهترین عدد
(متن تکراری) ۵۰ prefill / ۱۰ decode	انجام شد — فاز ۱	67.1 / 11.4
روی پرامپت واقعی ۱۰ decode	انجام شد — فاز ۲	13.35
۱۰۰ prefill / ۲۰ decode	نرسید decode انجام شد؛ prefill	105.6 / 18.09
(آرزوی فاز ۴) ۴۰۰ prefill	نرسید	اعتبارسنجی‌شده / ۲۳۶.۳ زنده 206.7
(هدف قدیمی) ۲۰ decode	نرسید	18.09 (mtp-18p4)

گام‌های منطقی بعدی بر اساس پروفایل فعلی: پروفایل‌گیری از کِرنل‌های نهایی prefill (پروفایل قبلی matmul dense/expert و سپس attention flash را گلوگاه نشان می‌داد)، آزمایش مقایسهٔ دقت با Q8E، و اگر هدف ۲۰ decode جدی ماند، کار روی نرخ پذیرش MTP (الان ~۷۶–۸۳٪ با cap=1).

۱۳. پیوست — بازتولید، فایل‌ها و سابقهٔ تغییرات

اجرای سرورها

سرور آزمایشی 8017:

```
cd /home/saeed/glm-tuning-20261011 && CTX=524288 setsid nohup ./run-prefill-validated.sh & > log 2>&1
```

سرور پایدار 8016:

`bash /home/saeed/glm-tuning-20261011/run-server.sh` (از `run-prefill-400.sh` با `NO_CACHE_PROMPT=1` برای بنچمارک کانونیکال بدون کش)

مستندات و شواهد در ورک‌اسپیس محلی

فایل	محتوا
RESULTS.md	روایت کامل فازهای ۱ و ۲ با جدول‌ها و تست‌ها
CHECKPOINT.md	تگ‌ها و رویدادهای شبکه، validated چک‌پوینت‌های
EXPERIMENT-100-20.md	دفتر آزمایش‌های فاز ۳ (پذیرفته‌ها و ردشده‌ها با اعداد)
prefill-final/PREFILL-400-STATE.md و PREFILL-RESULTS.md	زنده ablation + وضعیت نهایی فاز ۴
prefill-final/prefill-ablation-20261012.json	چهار ترکیبی ablation دادهٔ خام
validated-*.bundle (۸ ~۳۶فایل، MBهرکدام)	همهٔ چک‌پوینت‌های validated Git باندل‌های
cpu-tuning.patch، patch_moe.py، prefill-final/prefill-final.patch	پچ‌های سورس نسبت به مبنا
test_*.cpp (amx، concat، rows، gate، prefill)	سورس تست‌های عددی بایت‌دقیق
prefill-final/real-*.jsonl و real-*.jsonl	نتایج خام پرامپت واقعی هر مرحله
test-ui/ (server.py + dist/)	~/glm-test-ui در g11 سورس رابط کاربری — مستقر روی

سابقهٔ گیت این مخزن محلی

کامیت	موضوع
512e5a8	ایجاد مخزن: ورک‌اسپیس، tuning و بنچمارک‌ها UI
703844c	تکمیل UI: متریک، تفکر مدل، استریم MTP
88c9c5c	زندهٔ فلگ‌های ablation prefill
4c038df	کش پرامپت روی سرور آزمایشی + حالت حافظهٔ UI
2a7eb8a	خروجی بلند (۱۰۰K) ۵۱۲ کانتکست وK

منبع دیاگرام‌ها: نسخهٔ HTML هر چهار شکل mermaid در پوشهٔ report/mermaid-src/ نگهداری می‌شود و با Chrome headless رندر شده‌اند.